

E. Fable

ASTRA–7.0

Addendum with changes and new features

February 2012

MAX-PLANCK-INSTITUT FÜR PLASMAPHYSIK

GARCHING BEI MÜNCHEN

ASTRA–7.0

Addendum with changes and new features

E. Fable

November 2013

Introduction

Astra-7.0 is the natural continuation of the Astra concept, as brought to life and developed to the great extent that it is by the late Prof. Dr. G. V. Pereverzev¹ and Dr. Y. P. Yushmanov.

The reference user manual is still the IPP report 5/98 (February 2002), since the philosophy and main features of the code have been preserved. The main changes, which will be presented in detail in this addendum, have tried to address issues that in my opinion lead to an improvement of the code capabilities with respect to specific physical problems.

The choice of the numerical grid has been argument of many discussions, which resulted in considering the normalized grid as a possible alternative to the ρ -grid employed in Astra.

The complex problem of coupling dynamically an evolving plasma with internal transport processes on a free-boundary equilibrium background has been one of the latest challenges faced by us. Its solution would allow to perform full-tokamak-discharge simulations in Astra. The coupling scheme chosen ultimately is, in my opinion, the best possible, both in terms of stability and of physical consistency.

The inclusion of a theory-based momentum transport equation is something that Astra needed to complete its capability of full-channels transport analysis. In the new version, species temperatures, densities, and toroidal rotation velocities can be simultaneously evolved.

Additional packages (external codes, subroutines) have also been added, and described in this addendum.

Some of the material presented here is part of upgrades that were already present in Astra-6.2.1, as implemented by Prof. Pereverzev, however is not in the reference user manual: the IPC paradigm for parallel processes, the treatment of implicit equipartition, a novel numerical scheme to treat stiff transport problems, and some additional minor changes that will be presented in the following.

¹in memoriam

Contents

1	Normalized grid as the reference grid	5
1.1	Motivations for the change of grid	5
1.2	Description of the chosen grid	5
1.3	Grid convection terms when working with the x grid	7
1.4	Choice of the numerical scheme	8
2	Toroidal momentum transport equation	9
2.1	Derivation of the equation implemented in Astra-7.0	9
2.2	Evaluation of the radial current and consequences on momentum transport	10
2.3	Internal variables	14
2.4	Dedicated control parameters	15
3	Coupling with the 2D magnetic equilibrium	17
3.1	The Grad-Shafranov equation for a ψ -solver	17
3.2	The numerical scheme in details	19
3.2.1	The numerical scheme: parameter namelist	21
3.3	Choosing between free-boundary and prescribed boundary equilibria	22
3.3.1	The SPIDER parameter list	22
3.4	Free-boundary condition on FP equation	23
3.5	Choosing if to solve circuit equations and controller	23
3.6	Sign of toroidal field and toroidal current	24
3.7	PSIAX and PSIBO	25
3.8	Machine dependency	25
4	Parallel processes with IPC	27
4.1	Calling IPC from the model file	27
4.2	Setting up the subroutines to run the IPC processes	27
4.3	Calling subroutines in order	28

5	Numerical scheme for stiff transport problems	30
5.1	The scheme	30
5.2	Implicit equipartition	31
6	A Newton-scheme for fast convergence to an absolute steady-state	32
6.1	The scheme	32
6.2	New parameter	33
6.3	Some tricks	35
7	Additional packages available with Astra-7.0	36
7.1	TORBEAM for ECRH/ECCD calculations	36
7.2	Quasi-linear transport model TGLF	37
7.3	Quasi-linear transport model QuaLiKiz	37
7.4	TORIC for ICRF calculations	37
7.5	STRAHL for impurity transport calculations	37
7.6	NEO for neoclassical calculations	37
7.7	ASTRA restart files	37
8	Additional changes performed in Astra-7.0	39
8.1	New internal arrays	39
8.2	New formulas	40
8.3	Generic implicit equipartition	40
8.4	New definitions for external metric	40
9	Summary of changes implemented in Astra-7.0	42
10	Foreseen upgrades for next versions	44

1 Normalized grid as the reference grid

In this section it is described which grid has been implemented in the new version of the code. The form that the transport equations acquire when written in terms of this grid is also presented and discussed.

1.1 Motivations for the change of grid

In Astra-7.0 it has been decided to adopt another grid, which has advantages and disadvantages over the grid used in previous versions. The motivations for this choice stem from three considerations:

- 1) With the previous, ρ -grid, the number of grid points in the plasma region varies with the plasma linear dimension. This means that simulations which involve expanding plasmas with much different volume at the beginning and at the end of the expansion require a large number of grid points to maintain a sufficient resolution at every time step;
- 2) The boundary point does not fall on a pre-existing grid point, meaning that the last two grid points acquire a differential $\delta\rho$ which varies with time;
- 3) Coupling with equilibrium codes is extremely difficult and prone to numerical instabilities.

The main advantage of the ρ -grid is that grid-convective terms appear only for cases with time-dependent vacuum field $\dot{B}_0$. In case of static vacuum field, changes in plasma boundary do not affect internal grid points, which simplifies the equations. However, in the general case, the equations have the same form as they would appear on the normalized grid, where in the latter case $\dot{\Phi}_b$ appears as the convective drive.

1.2 Description of the chosen grid

From this version onwards, the fundamental grid on which quantities are computed is defined as:

$$x = \sqrt{\frac{\Phi}{\Phi_b}} \quad (1)$$

In terms of indexing, the main transport grid is numerically defined as ($j \in [1..NA1]$):

$$x_j = \frac{(j - 0.5)}{(NA1 - 0.5)} \quad (2)$$

where $NA1$ is the number of grid points in the plasma. The grid is thus equispaced. As such we have that $x_{NA1} = 1$ is always a grid point. The shifted grid on which fluxes and gradients are defined is given as:

$$\tilde{x}_j = \frac{(j - 1.)}{(NA1 - 0.5)} \quad (3)$$

This choice of indexing is useful if simulations are performed at given boundary values of the main quantity. For flux-driven simulations, it would be more appropriate and accurate to have $\tilde{x}_{NA1} = 1.$

To allow the user to choose between the two possibilities, i.e. having the actual plasma boundary on the main or on the shifted grid, an option has been introduced. The variable **FLXDR** can now be set to 0 or to 1 to this purpose, as seen in table 1.1 below.

Table 1.1 Choice for grid indexing label **FLXDR**

Name	Value	Description
FLXDR	0	The main grid $x = 1$ coincides with the plasma boundary
	1	The shifted grid $\tilde{x} = 1$ coincides with the plasma boundary

The variable **FLXDR** is a control parameter and can be added in the 'Control parameters' section of the `/equ/log/[model_file_name]`. If not provided explicitly, the default value is **FLXDR=0**.

IMPORTANT NOTE: A major difference with respect to the previous Astra-6.2.1 version is that now the number of grid points in the plasma, **NA1**, has to be given in input, instead of NB1, the latter being then a time-dependent quantity. In practice, the number of grid points in between the plasma boundary and the wall is a varying quantity. From the point of view of the INPUT experimental and model files to Astra, that is the only relevant change that the user should take care of. Every other detail, mapping from grid to grid etc. is taken care of inside Astra.

The grid variables are now part of the Astra arrays set, with the names:

Grid arrays

Name	Description	Units
XRHO	Main normalized grid	[]
SXHO	Shifted normalized grid	[]
RHO	Main grid in $\rho = \sqrt{\Phi/(\pi B_0)}$	[m]
SRHO	Shifted grid in $\rho = \sqrt{\Phi/(\pi B_0)}$	[m]

The ρ grid RHO is now defined simply as $\rho = \rho_b x$, and SRHO as $\tilde{\rho} = \rho_b \tilde{x}$.

1.3 Grid convection terms when working with the x grid

In the previous Astra versions, the use of ρ as the fundamental grid resulted in exact conservation of profiles for cases with vanishing transport fluxes and sources, made exception for cases with time-dependent vacuum toroidal field. The latter cases were treated in terms of 'adiabatic compression', a posteriori (i.e. after solution of the actual transport equations).

When transforming equations to have time derivatives done a constant x instead of constant ρ , a new convective term appears, in this form:

$$\left[\frac{\partial y}{\partial t} \right]_x - x \frac{\Phi_b}{2\Phi_b} \frac{\partial y}{\partial x} + [\text{transport}] = [\text{source}] \quad (4)$$

The second term on the LHS is the new grid convection term, proportional to the quantity Φ_b . Because this term depends on the solution of the equilibrium problem at two different time slices, it is now chosen to add this convective terms *a posteriori*, as was done before, like an 'adiabatic compression/expansion'. The new term has also been numerically implemented in the actual matrix which solves for the transport equations. However then it would require iterate at least once between transport and equilibrium.

To allow the user to choose if to solve the convective term inside or outside the equations, a new variable **ADCMPF** has been added to the Control parameters. The possible values are listed in the Table 1.2 below:

Table 1.2 Choice for adiabatic compression label **ADCMPF**

Name	Value	Description
ADCMPF	1 or -1	Adiabatic compression done in the transport equation, at the moment it is done explicit (ADCMPF=1), improved by iterating between transport and equilibrium (NITREQ > 1). Implicit scheme (ADCMPF=-1) will be implemented soon
	0	Adiabatic compression is performed outside of transport equation. Works only for FP!
	2	Adiabatic compression is neglected.
	(3)	(can then be done by equilibrium code, not recommended).

By default, **ADCMPF**=1. We note that neglect of adiabatic compression may not be too dramatic for kinetic profiles in general. Exception makes the poloidal flux ψ , which suffers from severe errors if no adiabatic compression is computed.

1.4 Choice of the numerical scheme

In the new version of Astra, new control parameters have been added, which allow the user to choose between several implemented numerical schemes to solve the transport equations.

These newly introduced variables are called **INUM*Ei***, where *i* can be 1,2,3, or 4. The different numbers refer to the quantities to be evolved:

- 1 - NE, F1, F2, ...;
- 2 - TE and TI;
- 3 - FP;
- 4 - UPAR;

The values that the INUM*Ei* can take are $k_1 k_2$, where k_1 and k_2 are two integer numbers. For example INUM1 = 21 means $k_1 = 2$ and $k_2 = 1$.

The meaning of the k_1, k_2 numbers is listed and detailed in the following table 1.3:

Table 1.3 Numerical scheme selection

Name	Description	Default
k_1	1	Scheme is fully explicit in time
	2	Scheme is fully implicit in time
	3	Scheme is Crank–Nicholson in time
k_2	1	Advection–diffusion is treated with centered difference in space
	2	The power–law scheme is employed
	3	The exponential scheme is employed

Default value is 22 for all INUM*Ei*, meaning that by default all transport equations are advanced fully implicitly in time, centered difference in space numerical schemes. The centered difference scheme is accurate for cases with small convection and large diffusion. Cases with large convection are better treated with the exponential or the power–law scheme. In future versions, it is possible that more sophisticated numerical scheme with gradient limiters will be implemented, however in general, cases with large convection are rarely encountered.

Notice that, if **INUM3** ≤ 0 , which has to be used with CU:AS; in the model file, then EQPF and EQFF (i.e. P' and FF') are to be prescribed by the user and are not computed in ASTRA. This modality is useful when performing equilibrium tests at constant P' , FF' .

2 Toroidal momentum transport equation

In this section the newly implemented transport equation to advance toroidal momentum is derived in details and the options to call it in the model file are presented.

The motivation that has pushed to add this important piece of physics stems from the fact that application of toroidal momentum transport theory with respect to experimental studies has rapidly expanded and it is now a very important part of the fusion research. In the previous Astra versions, it was possible to mock-up momentum transport via the *Fi* equations. However that has strong limitations and the physical representation of momentum transport through density equations is not accurate in most cases. That is why this equation has been added to the code.

The derivation presented here follows standard literature, from old classic derivations as in Hinton–Hazeltine review on tokamak plasma transport theory, to more recent as in V. Rozhansky *et al.* or in J. D. Callen *et al.*

2.1 Derivation of the equation implemented in Astra-7.0

The basic assumption behind the derivation of the toroidal momentum transport equation is macroscopic plasma quasi-neutrality:

$$\nabla \cdot \mathbf{J} = 0 \quad (5)$$

Its flux-surface ($\langle f \rangle \doteq \frac{\partial}{\partial V} \int f dV$) average reads

$$\langle J_{\Psi} R B_p \rangle = 0 \quad (6)$$

with J_r the net radial current of the bulk plasma, R the local major radius, and B_p the poloidal magnetic field.

The radial current density is given by non-ambipolar particle transport. In the case of a multi-species plasma this writes down as:

$$J_{\Psi} = e \sum_s Z_s \Gamma_s \quad (7)$$

with eZ_s the species charge, and Γ_s the cross-flux surface particle flux of species s .

2.2 Evaluation of the radial current and consequences on momentum transport

Each species s has its own momentum transport equation, that is the complete three-dimensional force balance:

$$\frac{\partial(m_s n_s \mathbf{U}_s)}{\partial t} + \nabla \cdot (\Pi_{c,s} + \Pi_{n,s} + \Pi_{a,s}) = e Z_s n_s (\mathbf{E} + \mathbf{U}_s \times \mathbf{B}) + \mathbf{F}_s + \mathbf{S}_s \quad (8)$$

The different quantities and terms are respectively: m_s the species mass, n_s the species density, $\Pi_{x,s}$ the species anisotropic stress tensor (c=classical, n=neoclassical, a=anomalous). Note that the anomalous part supposedly contains also the turbulent part of the Maxwell stress tensor. $\mathbf{E}, \mathbf{B}$ are the electric and magnetic fields, $\mathbf{F}_s$ the friction force, and $\mathbf{S}_s$ the externally applied torque (NBI for example).

We now multiply scalarly momtranspeq3d by $R\mathbf{e}_\phi$ and sum over all species:

$$\frac{\partial M_\phi}{\partial t} + \sum_s R\mathbf{e}_\phi \cdot \nabla \cdot (\Pi_{c,s} + \Pi_{n,s} + \Pi_{a,s}) = R\mathbf{e}_\phi \cdot (\mathbf{J} \times \mathbf{B}) + R \sum_s S_{\phi,s} \quad (9)$$

where $M_\phi = \sum_s m_s n_s R U_{\phi,s}$ is the plasma total toroidal angular momentum density. Notice that, for an axisymmetric system, $R\mathbf{e}_\phi \cdot (\nabla \cdot) \rightarrow \nabla \cdot (R\mathbf{e}_\phi \cdot)$, leading to:

$$\frac{\partial M_\phi}{\partial t} + \sum_s \nabla \cdot (R\mathbf{e}_\phi \cdot \Pi_s) = R\mathbf{e}_\phi \cdot (\mathbf{J} \times \mathbf{B}) + R \sum_s S_{\phi,s} \quad (10)$$

where $\Pi_s = \Pi_{c,s} + \Pi_{n,s} + \Pi_{a,s}$. On the other hand: $R\mathbf{e}_\phi \cdot (\mathbf{J} \times \mathbf{B}) = R J_\psi B_p$.

Before taking the flux-surface average, we rewrite momtranspeq3drephi2 in 'conservative-like' form:

$$\int_V \left[\frac{\partial M_\phi}{\partial t} + \sum_s \nabla \cdot (R\mathbf{e}_\phi \cdot \Pi_s) - R\mathbf{e}_\phi \cdot (\mathbf{J} \times \mathbf{B}) - R \sum_s S_{\phi,s} \right] dV = 0 \quad (11)$$

The volume integral spans inside a closed flux surface identified by a constant label Φ , which we choose as the toroidal flux enclosed by said surface. In that case the integral can be swapped with the time differential through the operation:

$$\int_V \frac{\partial M_\phi}{\partial t} dV = \left[\frac{\partial}{\partial t} \int_V M_\phi dV \right]_\Phi - \oint_\Phi M_\phi \mathbf{v}_\Phi \cdot \nabla V \frac{dS}{|\nabla V|} \quad (12)$$

By doing this, and differentiating then by Φ at constant time, we get the equation ($V' = \partial V / \partial \Phi$):

$$\begin{aligned} \frac{1}{V'} \frac{\partial}{\partial t} (V' \langle M_\phi \rangle) + \frac{\partial}{\partial V} \left\langle \left(R \sum_s \Pi_{\phi\Psi,s} - M_\phi \mathbf{v}_\Phi \cdot \frac{\nabla V}{|\nabla V|} \right) |\nabla V| \right\rangle \\ - \langle R J_\Psi B_p \rangle - \left\langle R \sum_s S_{\phi,s} \right\rangle = 0 \end{aligned} \quad (13)$$

Therefore, imposing condition (6), we end up with the relevant equation:

$$\begin{aligned} \frac{1}{V'} \frac{\partial}{\partial t} (V' \langle M_\phi \rangle) + \frac{\partial}{\partial V} \left\langle \left(R \sum_s \Pi_{\phi\Psi,s} - M_\phi \mathbf{v}_\Phi \cdot \frac{\nabla V}{|\nabla V|} \right) |\nabla V| \right\rangle = \\ = \left\langle R \sum_s S_{\phi,s} \right\rangle \end{aligned} \quad (14)$$

The equation can also be written in terms of the coordinate $\rho = \sqrt{\Phi / (\pi B_0)}$ that is used in ASTRA ($V' = \partial V / \partial \rho$), and $B_0 = B_{0v}$ is the reference **BTOR** field:

$$\begin{aligned} \frac{1}{V'} \frac{\partial}{\partial t} (V' \langle M_\phi \rangle) + \frac{1}{V'} \frac{\partial}{\partial \rho} \left[V' \left\langle R \sum_s \Pi_{\phi\Psi,s} |\nabla \rho| \right\rangle - V' \rho \frac{\dot{B}_0}{2B_0} \langle M_\phi \rangle \right] = \\ = \left\langle R \sum_s S_{\phi,s} \right\rangle \end{aligned} \quad (15)$$

or

$$\begin{aligned} \frac{1}{V'} \frac{\partial}{\partial t} (V' \langle M_\phi \rangle) + \frac{1}{V'} \frac{\partial}{\partial \rho} \left[V' R_\phi - V' \rho \frac{\dot{B}_0}{2B_0} \langle M_\phi \rangle \right] = \\ = \left\langle R \sum_s S_{\phi,s} \right\rangle \end{aligned} \quad (16)$$

Now, we have to express $M = \langle M_\phi \rangle$ in some practical way. Since $M_\phi = R \sum_s m_s n_s U_{\phi,s}$, we write $U_{\phi,s} = R \Omega_s$, where Ω_s is the angular rotation frequency.

We have to find a way to express the multispecies velocity, and to express the rotation as a flux function.

Let us start from (for a generic species): $\mathbf{U} = U_\parallel \mathbf{b} + \mathbf{U}_\perp$, and $\mathbf{U} = U_\phi \mathbf{e}_\phi + U_p \mathbf{e}_p$, where $\mathbf{U}_\perp = \frac{[\mathbf{E} - \nabla P / (Zen)] \times \mathbf{B}}{B^2}$. Note that $H = (E_r - \partial_r P / (Zen)) / (R B_p) = H_E + H_P$ (i.e. split in electric field and pressure contributions) is a flux function. Also, $U_p / B_p = K(\Psi)$ is a flux function.

So, we get:

$$U_\parallel = KB + \frac{I}{B} (H_E + H_P) \quad (17)$$

and

$$U_\phi = KB_\phi + R(H_E + H_P) \quad (18)$$

or

$$IH_E = U_\parallel B - IH_P - KB^2 \quad (19)$$

Note that $H = H(\Psi)$, i.e. is a flux function, since $H(\Psi) = - \left[\frac{\partial \Phi_0}{\partial \Psi} + \frac{1}{Z_{en}} \frac{\partial P}{\partial \Psi} \right]$.

Let us define

$$u_\parallel = \langle U_\parallel B \rangle / B_0 = K \frac{\langle B^2 \rangle}{B_0} + \frac{I}{B_0} (H_E + H_P) \quad (20)$$

and

$$IH_E = B_0 u_\parallel - IH_P - K \langle B^2 \rangle \quad (21)$$

so that:

$$U_\phi = R \frac{B_0}{I} u_\parallel + KI \left(\frac{1}{R} - R \frac{\langle B^2 \rangle}{I^2} \right) \quad (22)$$

Then:

$$\Omega = \frac{B_0}{I} u_\parallel + KI \left(\frac{1}{R^2} - \frac{\langle B^2 \rangle}{I^2} \right) \quad (23)$$

This last equation allows to relate the code result for $u_\parallel$ to the experimental measurement of Ω at some location.

So, we have to compute $M = \langle M_\phi \rangle$, which, neglecting poloidal variations of the density (this will be upgraded in a future version), can be written as:

$$\langle R^2 \Omega \rangle = \frac{\langle R^2 \rangle B_0}{I} u_\parallel + KI \left(1 - \frac{\langle R^2 \rangle \langle B^2 \rangle}{I^2} \right) \quad (24)$$

In addition, for each impurity species $s \neq D$, the neoclassical formulas provide $\Delta u_{\parallel, Ds} = u_{\parallel}^D - u_{\parallel}^s$. So that we can write:

$$\sum_s m_s n_s u_{\parallel}^s = \rho_m u_{\parallel} - \sum_s m_s n_s \Delta u_{\parallel, Ds} \quad (25)$$

with the total mass density $\rho_m = \sum_s m_s n_s$. We also define $\delta_{\text{neo}} = \sum_s m_s n_s \Delta u_{\parallel, D_s}$, and $\sigma_{\text{neo}} = \sum_s m_s n_s K_s$.

We can then rewrite the average toroidal mass flow as:

$$\langle M_\phi \rangle = \Upsilon_0 u_{\parallel} + \Upsilon_1 + \Upsilon_2 \quad (26)$$

with

$$\begin{aligned} \Upsilon_0 &= \rho_m \frac{\langle R^2 \rangle B_{0v}}{I} \\ \Upsilon_1 &= -\frac{\langle R^2 \rangle B_{0v}}{I} \delta_{\text{neo}} \\ \Upsilon_2 &= I \left(1 - \frac{\langle R^2 \rangle \langle B^2 \rangle}{F} \right) \sigma_{\text{neo}} \end{aligned} \quad (27)$$

Note that in the limit of zero poloidal velocity and infinite friction, we can simplify $\Upsilon_1 = \Upsilon_2 = 0$.

The choice of $u_{\parallel}$ as the variable to be solved for is that the parallel direction is natural for plasma transport, as such theory-based transport model provide transport of parallel velocity. Infact, the term $\left\langle R \sum_s \Pi_{\phi\Psi, s} |\nabla \rho| \right\rangle$ can be represented by considering that:

$$\Pi_{\phi\Psi, s} \rightarrow \Pi_{\phi\Psi, s} + \frac{B_\phi}{B} \Pi_{\parallel\Psi, s} - \frac{B_p}{B} \Pi_{y\Psi, s} + m_s U_{\phi, s} \Gamma_s \quad (28)$$

where the parallel and binormal stress-tensors represent viscous terms at a fixed Φ surface. Also here Γ_s is the particle flux. Also:

$$R_\phi \rightarrow R_\phi + R_{\parallel} - R_y + m_s U_{\phi, s} \Gamma_s \quad (29)$$

The parallel stress tensor is in general given by (the species s is intended, and also at the moment the residual stress terms are not included):

$$\left\langle R \frac{B_\phi}{B} \Pi_{\parallel\Psi} |\nabla \rho| \right\rangle = -mnR_0^2 \langle |\nabla \rho|^2 \rangle \left[\chi \left(\frac{\partial H_e}{\partial \rho} + \frac{1}{I} \frac{\partial (IH_p)}{\partial \rho} \right) + \chi_\theta \frac{b_2}{I} \frac{\partial K}{\partial \rho} - V H_e - V_\theta \frac{b_2}{I} K - V_d H_p \right] \quad (30)$$

where χ, χ_θ are the turbulent viscosities of the toroidal and poloidal flow respectively, V, V_θ, V_d are the pinch of electric, poloidal, and diamagnetic flows. $b_2 = \langle B^2 \rangle$. The expression can be rewritten upon substituting using formulas (21), and summing over species:

$$\left\langle R \frac{B_\phi}{B} \Pi_{\parallel\Psi} |\nabla \rho| \right\rangle = \frac{R_0^2 B_0}{I} \langle |\nabla \rho|^2 \rangle \left[-a_1 \frac{\partial u_{\parallel}}{\partial \rho} + a_2 u_{\parallel} + a_3 \delta'_{\text{neo}} + a_4 \delta_{\text{neo}} + a_5 \sigma'_{\text{neo}} + a_6 \sigma_{\text{neo}} + a_7 \delta_{d, \text{neo}} \right] \quad (31)$$

with the following definitions:

$$\begin{aligned}
a_1 &= \rho_m \chi \\
a_2 &= \rho_m \left(V + \chi \frac{\partial \log I}{\partial \rho} \right) \\
a_3 &= \chi \\
a_4 &= - \left(V + \chi \frac{\partial \log I}{\partial \rho} \right) \\
a_5 &= \frac{b_2}{B_0} (\chi - \chi_\theta) \\
a_6 &= \frac{b_2}{B_0} \left(V_\theta - V - \chi \frac{\partial \log I}{\partial \rho} + \chi \frac{\partial \log b_2}{\partial \rho} \right) \\
a_7 &= \left(V_d - V - \chi \frac{\partial \log I}{\partial \rho} \right) \\
\delta'_{\text{neo}} &= \sum_s m_s n_s \frac{\partial \Delta u_{\parallel, Ds}}{\partial \rho} \\
\sigma'_{\text{neo}} &= \sum_s m_s n_s \frac{\partial K_s}{\partial \rho} \\
\delta_{d, \text{neo}} &= \sum_s m_s n_s \frac{I H_P}{B_0}
\end{aligned} \tag{32}$$

Of course all of the above assumes that the transport coefficients are the same for all ion species (this will be improved in a future upgrade).

The binormal stress tensor can be kept as a whole since in general cannot be represented in terms of diffusion and convection.

2.3 Internal variables

The parallel velocity to be solved for would be:

$$\text{UPAR} = u_{\parallel} \tag{33}$$

The main ions diffusivity, pinch, and residual stresses (assumed to be the same for all ion species at the moment) are given in the table below:

$$\begin{aligned}
\text{XUPAR} &= \chi \\
\text{XUPAP} &= \chi_\theta \\
\text{CNPAR} &= V
\end{aligned}$$

$$\begin{aligned}
\text{CNPAD} &= V_d \\
\text{CNPAP} &= V_\theta \\
\text{RUPAR} &= R_\parallel \\
\text{RUPYR} &= R_y \\
\text{RUPFR} &= R_\phi \\
\text{TTRQ} &= \left\langle R \sum_s S_{\phi,s} \right\rangle
\end{aligned} \tag{34}$$

Table 2.1 Variables (arrays) related to the momentum transport equation

Name	Units	Definition	Description
MRHO	$\text{kg}/\text{m}^3/(10^{19} m_{\text{proton}})$	ρ_m	Total mass density $\rho_m = \sum_s m_s n_s$
DLNEO	$\text{kg}/(\text{m}^2 \text{ s})/(10^{19} m_{\text{proton}})$	δ_{neo}	$\delta_{\text{neo}} = \sum_s m_s n_s \Delta u_{\parallel,Ds}$
DLNEOD	$\text{kg}/(\text{m}^3 \text{ s})/(10^{19} m_{\text{proton}})$	δ'_{neo}	$\delta'_{\text{neo}} = \sum_s m_s n_s \frac{\partial \Delta u_{\parallel,Ds}}{\partial \rho}$
SGNEO	$\text{kg}/(\text{m}^2 \text{ s T})/(10^{19} m_{\text{proton}})$	σ_{neo}	$\sigma_{\text{neo}} = \sum_s m_s n_s K_s$
SGNEOD	$\text{kg}/(\text{m}^3 \text{ s T})/(10^{19} m_{\text{proton}})$	σ'_{neo}	$\sigma'_{\text{neo}} = \sum_s m_s n_s \frac{\partial K_s}{\partial \rho}$
DDNEO	$\text{kg}/(\text{m}^2 \text{ s})/(10^{19} m_{\text{proton}})$	δ_d	$\delta_d = \sum_s m_s n_s I H_{p,s} / B_0$
UPAR	m/s	$u_{\parallel}$	Parallel velocity as defined in equation (20)
XUPAR	m^2/s	χ	Radial diffusivity of parallel momentum
XUPAP	m^2/s	χ_θ	Radial diffusivity of poloidal momentum
CNPAR	m/s	V	Radial pinch of parallel momentum
CNPAD	m/s	V_d	Radial pinch of diamagnetic momentum
CNPAP	m/s	V_θ	Radial pinch of poloidal momentum
RUPAR	$1/\text{s}^2$	$R_\parallel$	Parallel Reynold stress $R_\parallel = \frac{\sum_s \langle R \frac{B_\phi}{B} \Pi_{\parallel \Psi,s} \nabla \rho \rangle}{(10^{19} m_{\text{proton}})}$
RUPYR	$1/\text{s}^2$	R_y	Perp Reynold stress $R_y = \frac{\sum_s \langle R \frac{B_p}{B} \Pi_{y \Psi,s} \nabla \rho \rangle}{(10^{19} m_{\text{proton}})}$
RUPFR	$1/\text{s}^2$	R_ϕ	Toroidal Reynold stress $R_\phi = \frac{\sum_s \langle R \Pi_{\phi \Psi,s} \nabla \rho \rangle}{(10^{19} m_{\text{proton}})}$
TTRQ	$\text{N}/\text{m}^2/(10^{19} m_{\text{proton}})$	$\langle R \sum_s S_{\phi,s} \rangle$	Toroidal input torque

Notice that MRHO, DLNEO, SGNEO, XUPAR, XUPAP, CNPAR, CNPAD, CNPAP, RUPAR, RUPYR, RUPFR, TTRQ, DLNEOD, DDNEO, SGNEOD have to be provided by the user in the model file, while UPAR is the evolved quantity.

2.4 Dedicated control parameters

Table 2.2 Control parameter **IPROT**

Name	Value	Description
IPROT	0	Include poloidal rotation terms
	1	Neglect poloidal rotation terms

By default, **IPROT = 0**.

3 Coupling with the 2D magnetic equilibrium

In this section it is detailed the scheme which Astra-7.0 uses to couple with a generic Grad-Shafranov solver, be it prescribed or free-boundary. The main idea behind the coupling scheme is: the use of normalized grid x together with the current diffusion being solved for ψ , suggest to devise a scheme in which ψ becomes the fundamental grid. This scheme is somewhat different from what is done in so-called q -solvers, which perform iterations around the invariance of the safety factor q . Other schemes use the parallel current $j_{||}$ as the quantity that directs the iterations. The common experience is that both q -solvers and $j_{||}$ -solvers are prone to numerical instabilities, hampering the ease of simulating disparate scenarios.

In the following it is explained why, what is called later on, ψ -solver is better suited to guarantee stability and accuracy by resorting to not so complicated numerical scheme.

3.1 The Grad-Shafranov equation for a ψ -solver

The starting point is the Grad-Shafranov equation (GSE):

$$\Delta^* \Psi = -4\pi^2 \mu_0 R^2 \frac{\partial P}{\partial \Psi} - 4\pi^2 F \frac{\partial F}{\partial \Psi} + \dots (\text{coils}) \dots \quad (35)$$

where $\Psi(R, Z)$, $\psi(x)$, and one requires that $\Psi(x(R, Z)) = \psi(x)$ on closed flux surfaces. $F = RB_\phi$ is the usual poloidal current integral.

A useful relation is found between F and derivative of ψ over x :

$$\begin{aligned} F(x) &= \frac{2\pi}{g_3} \Phi_b H \hat{q} \\ H &= \frac{\partial \psi}{\partial V} \\ \hat{q} &= 2x \frac{\partial x}{\partial \psi} \end{aligned} \quad (36)$$

where $g_3 = \left\langle \frac{1}{R^2} \right\rangle$. Notice that in the definition of F there are two unknowns at the level of transport: $H(x)$, and Φ_b . These two quantities cannot be determined solely from 1D radial transport processes. However they are essential to compute both the poloidal (and thus the parallel) current, and to compute the radial coordinate $\rho = \rho_b x$ that is actually used in the transport equations. The user can also prescribe the value of F at the plasma/vacuum interface, i.e. $F_b = F_{\text{vac}}$. If there are no poloidal currents flowing outside the separatrix, then $F_b = R_0 B_{\phi,0}$, i.e. the vacuum

constant value. The value of F_{vac} *per se* does not influence the equilibrium solution. However it influences the value of ρ_b .

Note also that $P' = \partial P / \partial \Psi$ is a known function of x .

Upon substitution of definition (36) into the GSE we get (external coil currents are not mentioned from now on):

$$\Delta^* \Psi = -4\pi^2 \mu_0 R^2 \frac{\partial P}{\partial \Psi} - 16\pi^4 \Phi_b^2 \frac{\hat{q}}{g_3} H \frac{\partial}{\partial \Psi} \left(\frac{\hat{q}}{g_3} H \right) \quad (37)$$

The flux-surface average of equation (37) looks like:

$$\frac{\partial}{\partial V} (g_2 H) = -4\pi^2 \mu_0 \frac{\partial P}{\partial \Psi} - 16\pi^4 \Phi_b^2 \hat{q} H \frac{\partial}{\partial \Psi} \left(\frac{\hat{q}}{g_3} H \right) \quad (38)$$

with $g_2 = \left\langle \frac{|\nabla V|^2}{R^2} \right\rangle$.

Or:

$$H \frac{\partial}{\partial x} (g_2 H) = -4\pi^2 \mu_0 \frac{\partial P}{\partial x} - 16\pi^4 \Phi_b^2 \hat{q} H \frac{\partial}{\partial x} \left(\frac{\hat{q}}{g_3} H \right) \quad (39)$$

By defining $P_x = \partial P / \partial x$, and the additional quantities: $y = H^2 / 2$, $A = g_2 + 16\pi^4 \Phi_b^2 \hat{q}^2 / g_3$, $B = 2(g_{2,x} + 16\pi^4 \Phi_b^2 \hat{q}(\hat{q}/g_3)_x)$, and $C = -4\pi^2 \mu_0 P_x$, we can compact the equation in:

$$A y_x = -B y + C \quad (40)$$

which solution is easily computed as:

$$y(x) = y_0 e^{-[B](x)} + e^{-[B](x)} \int_0^x e^{[B](x')} C(x') dx' \quad (41)$$

where here we temporarily employ the notation $[B](x) = \int_0^x B dx$.

The boundary condition $y(x=1) = y(1)$ is given in terms of known quantities:

$$y(1) = \frac{1}{2} \left(\frac{F_{\text{vac}} g_3(1)}{2\pi \Phi_b \hat{q}(1)} \right)^2 \quad (42)$$

which allows to compute the solution to equation (41):

$$y(x) = \frac{y(1) - e^{-[B](1)} \int_0^1 e^{[B](x')} C(x') dx'}{e^{[B](1)}} e^{-[B](x)} + e^{-[B](x)} \int_0^x e^{[B](x')} C(x') dx' \quad (43)$$

Having this solution, we can finally solve for $H(x) = \sqrt{2y(x)}$.

The knowledge of $H(x)$ allows to compute the plasma volume $V(x)$ as:

$$V(x) = \int_0^x \frac{2x}{H\hat{q}} dx \quad (44)$$

On the other hand, the total plasma volume $V_b = V(1)$ cannot be solved for by the flux–surface averaged GSE, since it depends on the exact last closed flux surface shape.

What then happens is that equation (45) is actually a non–linear equation for Φ_b :

$$V_b(\Phi_b) = \int_0^1 \frac{2x}{H(\Phi_b)\hat{q}} dx \quad (45)$$

Having now found both $H(x)$ and the Φ_b consistent with $\psi(x)$ and with the plasma volume V_b , we can solve the 2D problem, equation (37), with boundary condition $\Psi_\infty = 0$. The 2D problem is solved in this form now:

$$\Delta^*\Psi = -4\pi^2\mu_0 \left(R^2 - \frac{1}{g_3} \right) \frac{\partial P}{\partial \psi} + \frac{1}{g_3} \frac{\partial}{\partial V} (g_2 H) \quad (46)$$

which manifestly shows that the 2D problem reduces to computation of just flux–surfaces shape instead of usual shape+labeling.

Notice that in the GSE equation, on the right–hand side, the quantity $H(x)$ has required both g_2, g_3 for its solution. However, both g_2, g_3 are also solutions of the 2D GSE equation. To converge these two metric coefficients, iterations are necessary. Once convergence is reached, it is possible to check that the flux jump inside the plasma coincides with the flux jump from current diffusion equation, i.e. $\Psi_b - \Psi_0 = \psi_b - \psi_0$.

In cases of free–boundary computations, the plasma volume V_b can also change during iterations.

3.2 The numerical scheme in details

Astra produces input profiles to the equilibrium coupling schemes both $\psi(x)$ and $P(x)$. x is here the main Astra grid. After entering the interface for the coupling the two profiles are preinterpolated on what will be used as equilibrium grid X , equispaced from 0 to 1. As such, the iteration scheme sees entering profiles $\psi(X), P(X)$. Note that usually $N_X < N_x$ to speed up equilibrium computations.

Then the coupling scheme proceeds as follows (given for the general case of free–boundary equilibria):

- 0) $\psi(X), P(X)$ enter in the iteration process;
 - 1ext) External iteration $j_{\text{ext}} = 0$: initial guesses for g_2, g_3, Φ_b, V_b are set;
 - 2ext) External iteration $j_{\text{ext}} = j_{\text{ext}} + 1$: update $g_2^{j_{\text{ext}}}, g_3^{j_{\text{ext}}}, V_b^{j_{\text{ext}}}$;
 - 1int) Internal iteration $i = 0$: uses last computed Φ_b as initial guess.
 - 2int) Internal iteration $i = i + 1$: solve equation for $H(x)^{i, j_{\text{ext}}}$ given $g_2^{j_{\text{ext}}}, g_3^{j_{\text{ext}}}, \Phi_b^{i, j_{\text{ext}}}$.
- Compute temporary volume V_b^i .
- 3int) If $V_b^i \neq V_b^{j_{\text{ext}}}$ outside tolerance, adapt $\Phi_b^{i, j_{\text{ext}}}$ (done with bisection method), and go back to 2int).
 - 4int) $\Phi_b^{i, j_{\text{ext}}}$ has converged $\rightarrow$ compute FF' ;
 - 3ext) Call generic Grad-Shafranov solver with inputs P' and FF' as functions of $\psi_n = (\psi - \psi_0)/(\psi_b - \psi_0)$. Returns outputs g_2, g_3 and new plasma volume V_b ;
 - 4ext) Check relative error between old and new metric coefficients. If over tolerance, go back to 2ext);
 - 1) Compute additional quantities requested by Astra;
 - 2) Interpolate all outputs from equilibrium grid X to transport grids $x, \tilde{x}$ accordingly.

After one such procedure, the program has performed i iterations on 1D equation and j_{ext} iterations of 2D calculations. The output metric coefficients are self-consistent with given boundary or external coil currents and with current diffusion ψ .

Since all passages involve solution of parabolic equations on the same grid, no instability is present. If centered differencies are used, as in our case, spurious oscillations (pure imaginary Fourier modes) can be present. However these are dealt with very easily by means of under-relaxation. In the present version, old and new values are weighted 50 % each to compute the updated value. This factor 0.5 is fixed for all computations and as such does not constitute a mean to force convergence, but simply to stabilize numerical oscillations, retaining the 2nd order accuracy of the numerical solution of the equations. Our experience is that one needs to under-relax only g_2 and V_b , which are the most sensisitive quantities to changes in plasma internal profiles.

Notice also that we kept this procedure very modular, such that any GS solver that uses P' and FF' as inputs can be plugged into the iteration scheme.

3.2.1 The numerical scheme: parameter namelist

In the directory *exp/equ/(MACHIN)/* (see below for the meaning of this) there can be a file called *namelist.txt* that can be changed to try different numerical parameters. Here is the list with some example values (default values used by the code are optimized for basic calculations):

```

0.002      ! time from which to fix P' and FF' (ignores current diffusion of ASTRA)
0          ! fix P' and FF' (ignores current diffusion of ASTRA)
5          ! not used
0          ! if value=1 writes an information file for testing equilibrium code standalone
0          ! 0 - do iterations always, 1 - does iterations only at first FBE call, -1 - does iterations always
           except at first FBE call (recommended value is 0)
3          ! numerical scheme for stabilization of numerical instabilities, 3 is recommended value
3          ! hard-wired parameters, leave 3
0          ! write files with diagnostic informations if value = 1
0          ! leave 0
0.5        ! under-relaxation factor
0.1        ! under-relaxation factor for metric coefficients correction
0.2        ! under-relaxation factor for metric coefficients derivatives
1.E+2      ! not used, do not change
1.         ! not used, do not change
10000      ! maximum number of iterations allowed
1          ! maximum number of iterations to be performed (if converges exits before of course). This
           depends on the time step. If time step is large, probably better to do some iterations
2          ! interpolation routine: 1 is quadratic, 2 is cubic splines
4          ! interpolation method for flux-surfaces to rectangular grid: 4 is bi-quadratic
1.E-9      ! tolerance on magnetic flux at boundary
1.E-5      ! tolerance on metric coefficients variation
1.E-5      ! tolerance on plasma volume variations
1.E-8      ! tolerance on metric coefficients derivative variation (not used)
0          ! if 0, does not do iterations if refit currents. 1 to do iterations when refitting currents
0          ! if 0 does not overwrite coil.dat with refitted currents

```

3.3 Choosing between free-boundary and prescribed boundary equilibria

A new control parameter **ITFBE** has been added, which can have values and meaning as explained in the following table 3.1.

Table 3.1 Choice of equilibrium solution with **ITFBE**

Name	Value	Description
ITFBE	-1	Equilibrium is solved in prescribed boundary mode
	> 0.0	Equilibrium is solved in free-boundary mode from TIME > ITFBE

By default, **ITFBE**=-1.

Another variable **IFBEG** regulates the choice between rectangular grid with **IFBEG=0** and adaptive grid with **IFBEG=1**. Notice that this choice has also to be set in the equilibrium code as well otherwise some possible incompatibility problems will arise.

3.3.1 The SPIDER parameter list

In the directory *exp/equ/(MACHIN)/* (see below for the meaning of this) there can be a file called *namelist - spider.txt* that can be changed to try different numerical parameters. Here is the list with some example values (default values used by the code are optimized for basic calculations):

```

0      ! print info on screen from SPIDER
0      ! choose k _ grid (0 for rectangular, 1 for adaptive)
1.0d-7    ! tolerance on fixed boundary calculations
1.0d-6    ! tolerance on circuit equations
1      ! key _ plc: 1 to fix IPL into SPIDER, 0 to allow IPL from SPIDER to be used as actual
current
4      ! number of toroidal harmonics for TORIC
0      ! if 1, save file for TORIC
0      ! if 1, save file for TORBEAM
0      ! if 1, save file for STRAHL
0      ! if 1, save file for EQDSK
3      ! number of fourier modes for STRAHL

```

0 ! if $\zeta > 0$, save respective fort. file with some diagnostic data

3.4 Free-boundary condition on FP equation

In addition, the possibility of imposing an alternative boundary condition to the current diffusion equation has been made available. The new control parameter **ITFBP** allows to choose this option. Its value are given in table 3.2:

Table 3.2 Choice of boundary condition with **ITFBP**

Name	Value	Description
ITFBP	0	No effect (i.e. uses the ASTRA b.c., either prescribed I_p or prescribed ψ_b)
	-1	Boundary condition is $\psi_b + \left[G \frac{\partial \psi}{\partial \rho} \right]_b = \psi_{\text{ext}}$

By setting **ITFBP**=-1, the boundary condition $\psi_b + \left[G \frac{\partial \psi}{\partial \rho} \right]_b = \psi_{\text{ext}}$ is used, where the left-hand-side is numerically discretized and the auxiliary values G, ψ_{ext} are provided by the equilibrium code from a free-boundary equilibrium computation. The exact definitions are:

$$G = \frac{1}{l_b} \oint \oint \text{Green}(\vec{r}, \vec{r}') \left| \frac{\nabla \rho}{R} \right|(\vec{r}') dl' \quad (47)$$

$$\psi_{\text{ext}} = \mu_0 \sum_j \frac{1}{l_b} \oint \text{Green}(\vec{r}_j, \vec{r}') I_j dl'$$

where "Green" is the Green function of the Grad-Shafranov Laplacian operator and I_j are the external conductors currents, and l_b is the LCFS perimeter length. The value of $\psi_b, \psi_{\text{ext}}, G$ in ASTRA are recorded in the scalar variables **PSIFB**, **PSIEXT**, **PSPLEX**. The default value of the b.c. control variable is **ITFBP**=0.

3.5 Choosing if to solve circuit equations and controller

Additional control parameters **ICIRCQ** and **IPCTRL** have been introduced. The possible options are:

Table 3.3 Choice of values for **ICIRCQ** and **IPCTRL**

Name	Value	Description
ICIRCQ	0	Does not run circuit equations in equilibrium code
	1	Runs circuit equations in equilibrium code
IPCTRL	0	Does not run controller, coil.dat untouched, no refit
	1	Runs controller, and refit coil-currents at first time step
	-1	Runs controller, and solves FB-GSE at first time step, given coil currents
	-2	No controller, do not refit coil-currents at first time step, CCOIL goes into coil.dat
	-3	No controller, do not refit coil-currents at first time step, CCOIL goes into coil.dat
	-4	No controller, refit coil-currents at first time step, coil.dat untouched
	-5	No controller, refit coil-currents at first time step, CCOIL into coil.dat

Notice that if IPCTRL is non-integer, in particular if written as IPCTRL=X.1 (where X is a value as in the table), then skips internal ASTRA iterations of NITREQ at first free-boundary call.

Notice also that if IPCTRL is such that CCOIL overwrites coil.dat, then one has to create, in the */exp/equ/(MACHINE)/* a directory called *req2d - input* where a file called *coil - ref.dat* should be present. This file has the same structure as coil.dat, and will be used as a template to create the coil.dat using CCOIL values for the coil currents.

3.6 Sign of toroidal field and toroidal current

From Astra-7.0 on, it is explicitly stated that BTOR, IPL, and related parameters (CU, MU, etc.) have to be given with positive sign with respect to the reference coordinate system (R, Z, ϕ) , or equivalently (ψ, θ, ϕ) . The sign as in the real experimental scenario to be simulated, has to be given separately with the two new control parameters **SGNIP** and **SGNBT**, as in table 3.3.

Table 3.4 Sign of reference toroidal current and toroidal field

Name	Value	Description
SGNIP	1	Current flows into positive ϕ direction
	-1	Current flows into negative ϕ direction
SGNBT	1	Toroidal field is positive in ϕ direction
	-1	Toroidal field is negative in ϕ direction

The signs will be taken into account in those parts of the code for which the direction of the field and current is relevant for the physical subsystem under study. The default values are **SGNIP**=1 and **SGNBT**=1.

3.7 PSIAX and PSIBO

Two new output variables have been added to Astra. **PSIAX** is the value of ψ at $x = 0$, exactly on the magnetic axis. Since the axis is not a grid point, **PSIAX** is obtained with quadratic extrapolation from the first three grid points. **PSIAX** is also used to compute the normalized poloidal grid properly.

The other variable **PSIBO** is the value of ψ at $x = 1$.

3.8 Machine dependency

To allow trating multiple machines (e.g. AUG, ITER, JET, etc...), a new variable has been added to ASTRA. The variable is called **MACHIN** and it is a 4-character string that has to be assigned by putting the name of the machine in the first four characters of the experimental file list. So if one wants to choose 'aug' the experimental file heading would read something like:

```
aug -AUG- #11110 t=100
Param—Time — Value — Error
```

For ITER it would look like

```
iter -ITER- #11110 t=100
Param—Time — Value — Error
```

The variable **MACHIN** is then assigned as 'aug' or 'iter'. The reason to do this is because, in the *exp/equ/* directory, the directory where equilibrium informations sit has the same name as trim(MACHIN) (i.e. without the eventual space in case of a 3-letters machine name). So for 'aug' there will be a *exp/equ/aug/* directory which ASTRA and the equilibrium code will use to put information on the equilibrium.

These trick allows to avoid having to recompile the code every time one wants to switch between machines.

4 Parallel processes with IPC

From Astra-6.2.1 on, it is possible to run several processes on independent memory segments, to speed up computations in cases where each radial grid point is independently considered.

As an example, suppose to run a transport simulation of electron temperature, for which the energy diffusivity is computed with a sophisticated model that requires some time for a single radial point computation. Said model is local, i.e. each radial grid point is computed independently of the others. In this case, the IPC paradigm is of help, since many radial grid points can be then run simultaneously which greatly speeds up the overall simulation.

4.1 Calling IPC from the model file

Suppose to have a subroutine *subp.f* which calls a physical model that computes at generic radii index j .

In the model file the IPC call to *subp.f* appears as a sequence of calls:

```
xpr/subp(j1,j2)&::;;
```

where $j1, j2$ are indexes of radial positions. As an example, with 121 radial grid points:

```
xpr/subp(1,40)&::;;
```

```
xpr/subp(41,81)&::;;
```

```
xpr/subp(82,121)&::;;
```

which means that the subp module is sent on three independent processes, the first of which computes radii from 1 to 40, the second from 41 to 81, and the third from 82 to 121. In this way, the computational time for 121 grid points is 1/3 than the usual computational time that would require the call

```
subp::;;
```

which solves all radii in one single process.

4.2 Setting up the subroutines to run the IPC processes

Running IPC requires some effort from the part of the user in setting up a bunch of subroutines that take care of appropriate variable assignment and calls. This is unfortunately not as straightforward as calling subroutines in the standard way.

The detailed explanation on how to set up these additional routines and the scheme of the IPC process is presented in the attached paper *IPC.pdf*.

4.3 Calling subroutines in order

In general, in all Astra versions, the order in which commands enter in the model file, is maintained in the compilation. In addition to this, from Astra-6.2.1 onwards, there is the possibility to explicitly tell the code to give higher or lower priority to specific subroutines.

Suppose to have five generic subroutines that here we call SUB1,SUB2,SUB3,SUB4,SUB5, where the number is arbitrary, does not represent priority. If in the model file they appear for example as:

```
SUB1:;;;  
SUB2:;;;  
SUB3:;;;  
SUB4:;;;
```

Means that they will be performed in the order in which they appear: 1,2,3,4. Now, by writing for example:

```
SUB1:;;;  
SUB2 < :;;;  
SUB3:;;;  
SUB4:;;;
```

The order would be: 2,1,3,4. SUB2 has acquired highest priority with the symbol < . The following instead:

```
SUB1:;;;  
SUB2 < :;;;  
SUB3 < :;;;  
SUB4:;;;
```

gives the order: 2,3,1,4. In this other example:

```
SUB1 > :;;;  
SUB2:;;;  
SUB3:;;;  
SUB4:;;;
```

the order is: 2,3,4,1. So SUB1 has lowest priority with the symbol $>$. In this example:

SUB1 $>$::::;

SUB2 $>$::::;

SUB3::::;

SUB4::::;

the order is then 3,4,1,2.

5 Numerical scheme for stiff transport problems

In this section it is discussed a scheme implemented in Astra to treat stiff transport problems.

5.1 The scheme

In most applications of tokamak transport, one encounters cases in which bulk plasma energy and particle transport is stiff. This term basically means that a small change in the transported quantity causes a large change in the flux.

Calling Q the flux of quantity T , this means that $Q(T, T')$ has a strong sensitivity in variations of T and T' .

As a simple example, say that transport of T satisfies the equation:

$$\frac{\partial T}{\partial t} + \frac{\partial Q}{\partial x} = 0 \quad (48)$$

and

$$Q = -D \left(\frac{\partial T}{\partial x} \right) \frac{\partial T}{\partial x} \quad (49)$$

In a linearly implicit scheme in time, this means that formally the solution is:

$$\frac{T^+ - T^-}{\delta t} - \frac{\partial}{\partial x} \left[D(T_x^-) \frac{\partial T^+}{\partial x} \right] = 0 \quad (50)$$

If solved in this way, and if the dependence of D on $T_x = \partial T / \partial x$ is rather strong (for example linear is sufficient), then the solution exhibits unphysical oscillations and stair-steps appear on the profile, which do not allow to smoothly follow the evolution.

In the paper [G. V. Pereverzev and G. Corrigan, Computer Physics Communications **8**, 579 (2008)], the Authors propose a scheme that efficiently mitigate this issue. The basic idea is to introduce an auxiliary diffusion coefficient D_s , where s stands for 'stiff'. The value of D_s can be a constant or a profile.

The modified equation looks then as:

$$\frac{T^+ - T^-}{\delta t} - \frac{\partial}{\partial x} \left[(D(T_x^-) + D_s) \frac{\partial T^+}{\partial x} - D_s \frac{\partial \log T^-}{\partial x} T^+ \right] = 0 \quad (51)$$

We can introduce an effective convection $V_s = D_s \frac{\partial \log T^-}{\partial x}$, such as to rewrite:

$$\frac{T^+ - T^-}{\delta t} - \frac{\partial}{\partial x} \left[(D(T_x^-) + D_s) \frac{\partial T^+}{\partial x} - V_s T^+ \right] = 0 \quad (52)$$

In the paper cited previously, it is proven that this method produces smooth evolution which, on average, is equivalent to solving the original equation. The numerical error introduced with this scheme with respect to the exact solution is also estimated in that paper.

In the code, a set of new array has been introduced to employ this scheme. The arrays are called **DVN**, **DVE**, **DVI**, respectively for electron density, electron temperature, and ion temperature. Their dimension is m^2/s as the other diffusivities. Their usage is as the following in the model file:

```
DVE=50.;
```

```
DVI=50.;
```

```
DVN=50.;
```

where 50. here is an example value, but it can be arbitrary, such as to sufficiently stabilize the diffusivity jumps but not too large to lose physics during transients for example. The actual criterion could be derived by comparing these artificial diffusivities with the physical diffusivity and the time scales of interest.

Notice that in the model file only the artificial diffusivities pertaining to the actually resolved equations have to be specified. Also DVE, DVI, etc. have to be defined AFTER the definitions of HE, XI, etc.!

5.2 Implicit equipartition

From Astra-6.2.1 onwards, there is the possibility to run electron and ion temperature equations with implicit equipartition. One needs to write as the following in the model file:

```
TE*:EQ;
```

```
TI*:EQ;
```

and in PE and PI there is no need to put PEICL.

6 A Newton-scheme for fast convergence to an absolute steady-state

In this section it is discussed a scheme implemented in Astra to converge to an absolute steady-state using the Newton-scheme.

6.1 The scheme

The scheme is supposed to solve the steady-state version of the transport equation, using Newton-like iterations. Precisely, we have a set of variables $y_i(x)$, where x is the radial coordinate and $i = 1..N$ with $y_1 = T_e, y_2 = T_i, y_3 = n_e, y_4 = F1, y_5 = F2, \dots$.

One selects a finite number of radial grid points $x_j, j = 1..M$, and the actual solution variable $g_{i,j}$, where at each chosen radial grid point j , $g_i = -d \log(y_i)/dx$.

Now, one can compute the transport sources $R_{i,j} = (P_e, P_i, S_n, \dots)$. As well as the transport fluxes: $T_{i,j} = (Q_e, Q_i, Q_n, \dots)$ as defined in ASTRA using diffusivities and convection velocities.

Obviously: $R = R(g)$ and $T = T(g)$ where the dependencies can be very very complex and non-linear. At steady-state one should find $R = T$. As such, we select the "figure-of-merit" $F = R - T$ as the variable to force to zero.

The scheme is thus employed to solve the simple implicit equation:

$$\frac{dF}{dt} = -F \quad (53)$$

where the time t here is not actual real time but just a parameter to identify evolving iterations in differential form. The implicit solution assuming previous iteration is known: F_0 , is:

$$F = \frac{F_0}{1 + dt} \quad (54)$$

So this is very straightforward and easy, however F is an implicit highly non-linear functions of the profiles, represented via the g s.

To explicit them, we use Fourier expansion around F_0 , that is $F = F_0 + J(g - g_0)$, where J is the Jacobian matrix and Jg is inteded as matrix multiplication operator.

Finally, the equation for the new value of the g s is simply:

$$g = g_0 - \frac{dt}{1 + dt} (J^{-1} F_0) \quad (55)$$

So the scheme is simply:

- 1) give some guess g_0
- 2) compute F_0
- 3) compute the Jacobian J and its inverse J^{-1}
- 4) compute the new g
- 5) if tolerance in $g - g_0$ is reached, exit, otherwise set $g_0 = g$ and go to 2)

It is obvious that the heavy part here is in computing J and the inverse. For example, if one computes transport of $T_e, T_i n_e$ with a transport grid of 7 points, there are a total of 21+1 (=22) calls of the ASTRA formulas and subroutines that define the transport coefficients and the sources, per iteration, plus the inverse matrix operation. This makes this scheme rather slow for several transport channels and many grid points.

An alternative scheme is proposed, which assumes that transport is dominated by diagonal diffusion of its own channel. In this case the iteration scheme is the following:

$$g = g_0 - \frac{dt}{1 + dt} \frac{F_0}{y_0 D_0} \quad (56)$$

where D_0 is the diagonal diffusion coefficient of the quantity under consideration and the radial grid point. As such, it is just a ratio of local quantities and not a matrix product. Moreover, it requires only 1 computation per iteration. Of course, the convergence in this case is not as fast as in Newton, in terms of number of iterations, but it is much faster.

As an example, for the electron temperature channel at some location j :

$$g_j = g_{0,j} - \frac{dt}{1 + dt} \frac{F_{0,j}}{n_{e,0,j} T_{e,0,j} \chi_{e,0,j}} \quad (57)$$

This alternative scheme is pretty fast and converges in less iterations than the standard time-dependent scheme, however if the transport matrix has non-negligible non-diagonal terms, it can take more iterations than the actual Newton scheme.

Note that the choice of dt is merely based on stability considerations, i.e. it must be such that the iterations do not oscillate or diverge.

6.2 New parameter

To switch from ASTRA time-dependent scheme to the Newton scheme, the new control variable **IPNWT** has been introduced. Here are the possible values:

Name	Value	Description
IPNWT	0.	Default: usual ASTRA time-dependent scheme
	-1.	ASTRA called with time-dependent scheme but it stops when tolerance is reached
	1	Simplified Newton scheme, ASTRA stops when tolerance is reached
	2	Simplified Newton scheme, ASTRA goes on (after a pause) with time-dependent scheme after tolerance is reached
	3	Simplified (0) or real (1) Newton scheme , it stops when tolerance is reached
	4	Simplified (0) or real (1) Newton scheme ,ASTRA goes on (after a pause) with time-dependent scheme after tolerance is reached
	5	First simplified Newton scheme, until a certain intermediate tolerance is reached, then goes on with real Newton scheme up to actual chosen tolerance

To select some numerical parameters, a new file is introduced and must be found in the local directory *dat/*

This file is called *newton_numerics.dat* and it contains one row with the following parameters (example):

number of transport grid points ; x0 ; xb ; number of channels ; tolerance ; isaw ; imode ; pown ;
powe ; urelax ; powsub ; idiag ; tolinterm ; forgetexp

5 0.2 0.9 4 1.e-6 1 1 0. 0. 1. 0. 0 1.e-1 1

Here is the meaning:

number of transport grid points: self-explanatory, equispaced between x0 and xb.

x0 : first grid point in normalized ρ

xb : last grid point in normalized ρ

number of channels : number of equations to be solved

tolerance : tolerance to be reached

isaw: if 0 q profile is natural, if 1 q profile is flattened to 1 inside $q = 1$

imode: if 0 runs simplified Newton scheme, if 1 runs real Newton scheme

pown: exponent of $1/tol^{pown}$ to call the Newton solver every tot iterations (suggested to leave = 0)

powe: exponent of $1/tol^{powe}$ to call the equilibrium solver every tot iterations (suggested to leave = 0)

urelax: under-relaxation coefficient (suggested to leave = 0)

powsub: exponent of $1/tol^{powsub}$ to call the postep.inc subroutines every tot iterations (important to avoid slowing down the code)

idiag: 0 to not write any diagnostics on screen, 1 to write

tolinterm: when IPNWT = 5, decides the tolerance at which to switch between simple Newton and real Newton

forgetexp: when 1, do not sync with experimental data; useful when for example parts of the profile have to be assigned iteratively (i.e. pedestal model)

6.3 Some tricks

The simplified Newton scheme always converge or at least is stable, even when the model file does not respect some logic, i.e. in terms of variable definitions, i.e. as in the usual time-dependent scheme.

The real Newton scheme diverges with nonsense results if the model file does not respect a certain logic. Here are the criterion:

- 1) There must not be any model that creates "jumps" in the profiles;
- 2) The definitions of the variable must follow consequently;
- 3) Avoid at all costs recursive definitions;
- 4) Heavy subroutines better go into postep.inc (symbol >);
- 5) Light subroutines that define transport coefficients or other things should be called with the symbol < , and define possibly the variable "work", because then they are called before the use of said "work";

7 Additional packages available with Astra-7.0

7.1 TORBEAM for ECRH/ECCD calculations

Recently, the TORBEAM code has been added to ASTRA through an interface subroutine *torba.f* that is found in the *sbr/* directory.

The executable files are to be extracted from the library *libtorbeam.a*, and the resulting *xxx.o* are to be added to the *user.a* archive of the Astra user.

To call TORBEAM just call it as a subroutine in the model file as:

```
TORBA(VAR1,VAR2)::::;
```

The outputs **VAR1,VAR2** are two arrays. **VAR1** is the ECRH power density deposition profile in [MW/m^3], while **VAR2** is the ECCD profile in [MA/m^2]. Note that the power profile is summed up over all gyrotrons, as well as the driven current profile.

At present TORBA can run up to 8 gyrotrons. Their specifications are to be prescribed in a file that has the same name as the experimental file, and has to be placed in the *exp/ecr/* directory. The structure of the ECRH file is the same as the NBI file explained in section 4, table 4.30. In the following table 6.1.1 the different parameters are explained for the ECRH file:

Table 6.1.1. Parameters describing each gyrotron

No.	Units	Description
1	[MW]	Gyrotron nominal power
2	–	Unused parameter
3	[GHz]	Frequency
4	[cm]	Major radius from where beam is launched
5	[cm]	Vertical position from where beam is launched
6	[deg]	Angle of poloidal injection (from LFS, positive for upward launching)
7	[deg]	Angle of toroidal injection (from LFS, positive for clockwise launching)
8–20	–	Unused parameters

The first parameter gives the power injected by each gyrotron. This quantity is usually time dependent, therefore, it is convenient to set it as a variable rather than as a number. For example, this variable can be a *ZRD* and its time evolution can be prescribed in the data file.

7.2 Quasi-linear transport model TGLF

The TGLF code [G. Staebler *et al.*, Phys. Plasmas **14**, 055909 (2007)] computes turbulence-driven fluxes for heat and particle transport via solution of fluid moments with kinetic closure fitted to match kinetic responses computed with linear gyrokinetic codes. The trapped and passing particle descriptions have been majorly upgraded with respect to its predecessor GLF23.

In principle different interfaces between Astra and TGLF are available, for both serial and parallel computations. They should be requested explicitly.

7.3 Quasi-linear transport model QuaLiKiz

7.4 TORIC for ICRF calculations

An interface to couple Astra to the TORIC code for ICRF heating computations has been developed. The routine is called *a2toric.f* and is part of the *sbr/* package. The TORIC library however has to be requested explicitly.

7.5 STRAHL for impurity transport calculations

Astra is also now coupled to the impurity transport code STRAHL, such that time-dependent computation can be performed with STRAHL providing impurity profiles, Z_{eff} profiles, and radiated power profiles.

The interface is called *A2STRAHL.f* and is part of the *sbr/* package. The STRAHL code itself has to be requested explicitly.

7.6 NEO for neoclassical calculations

7.7 ASTRA restart files

Sometimes ASTRA computations can be lengthy, and it is important to be able to save relevant data such that one can restart the simulation exactly as it has been saved.

This new routine is called *restart_astra.f* and is in the *sbr/* package. It can be called in the model file as:

```
RESTARTA::::;
```

as with all the other routines, it can be assigned a starting time, an ending time, and time intervals for execution, or keys to be pushed to activate it. Each time the routine is called, it will create

a new subdirectory in the *dat/* directory, the subdirectory being called *(model)_(expfile)_(index)*, where (model) is the model file name, (expfile) is the experimental file name, and (index) is just an index ordering the time slices.

Also, a new control parameter has been added, which is called **IRESTA**, that can have the following values:

Table 6.5.1 Choice of equilibrium solution with **IRESTA**

Name	Value	Description
IRESTA	0	Do not use any restart file
	<i>i</i> (integer > 0)	Use restart file with index <i>i</i>

By default, **IRESTA**=0.

If **IRESTA** > 0, then the restart file with index **IRESTA** is used. What the restart option does essentially is reading first the experimental file and model log file as they are, then reading the restart file and substituting all constants with the values as they are stored in the restart file. The constants include all variables and control/grid parameters. The simulation is then started from the **TIME** as stored in the restart file. As regards profiles, contrary to the usual Astra initialization, when using the restart file, the following profiles are substituted as they are stored in the restart file:

TE, NE, TI, F(1--12), FP, UPAR, MU, CU

Instead of using the values as given in the exp file. This allows to restart a simulation exactly from the state in which it was left before.

8 Additional changes performed in Astra-7.0

8.1 New internal arrays

Additional arrays have been added to ASTRA, that could be useful for post-processing and physics analysis. The new arrays are listed below, with definition:

Name ; Unit ; Definition

NMAIN [10^{19} m^{-3}] - Density of a "main ion" species, supposed to be the sum of all main ions (e.g. D, or D+T) (remember that NI is the sum of all THERMAL ions).

AREAT [m^2] - Area of the toroidal surface (area of a poloidal cut at fixed toroidal angle), of a flux surface.

PERIM [m] - perimeter of a flux surface, at a fixed toroidal angle.

These new variables can be used in formulas, subroutines, functions, etc. Notice that at the moment AREAT and PERIM are not compute by the 3-moment solver EMEQ.

As regards the description of the coils to be used in the free-boundary calculations, new arrays CCOIL and VCOIL are introduced (as well as CCOILX and VCOILX) In the experimental file, they should be defined as (in [kA] as units of currents):

NAMEXP CCOIL NTIMES 1 POINTS 13

0.

0.1

0.1

0.1

0.1

0.1

0.1

0.1

0.1
0.1
0.1
0.1
0.1
0.1

where the first value is the time point, and then 13 values follow for the 13 coils. The same structure has to be used for VCOIL (voltage in [V]).

8.2 New formulas

In fml/ there have been added two new formulas: *pei2* and *suzpei*. *pei2* accounts for implicit equipartition with impurities, where *suzpei* contains the formula of the summation over the thermal ion species. *suzpei* has to be in case removed by the user and re-created in the fml/ directory in case a different choice of the summation is wanted. *pei2* instead can be used as it is. To choose to run implicit equipartition with *pei2* the variable **IPROT** has to be set either = -2 or =2 (where the positive sign also sets in neoclassical contributions to parallel rotation in the UPAR equation).

For explicit equipartition, the formula *peicl2* can be found in /astra tools/fml/. It also uses *suzpei*.

8.3 Generic implicit equipartition

From version of 25 March 2020, the implicit equipartition term has to be defined in the model file with the variable PEIQI. For example:

PEIQI = PEI;

is equivalent to using the formula for PEI for the implicit equipartition as was done before, or one can use SUZPEI or PEI2 or others. If PEIQI is not defined, it is initialized to 0., as such there will be no implicit equipartition term.

8.4 New definitions for external metric

With respect to definitions given in section 5.3.3, the definition of the metric quantities used when NEQUIL = -1 is assigned (remember also that needs to go together with the variable assignment

of IPEQL (=1 by default)

...

NEQUIL = -1

IPEQL = 3

...

The new definitions are: SHIF = SHX; ELON=ELX; TRIA=TRX; VR=VRX;

DRODA=DRODAX; IPOL=IPOLX; G11=G11X;

G22=G22X; G33=G33X;

9 Summary of changes implemented in Astra-7.0

We summarize here the main differences and the new features with respect to the previous version Astra-6.2.1:

- The new grid is defined as $x = \sqrt{\Phi/\Phi_b}$. Grid convective terms have been added and can be treated either a posteriori (adiabatic compression/expansion), or iteratively (in the equations);
- A toroidal momentum transport equation has been added. It is solved for the parallel velocity. Poloidal rotation (Pfirsch-Schlueter terms) are not yet implemented;
- Coupling with the equilibrium codes has been made stable and consistent. It is valid for every code that uses P' and FF' as input for the equilibrium calculations;
- Already in Astra-6.2.1 (but not in the respective user manual though): implicit equipartition for Te, Ti evolution; possibility of running several independent processes in parallel with IPC; numerical scheme for stiff transport problems;
- Additional interfaces to packages are available in Astra: TORBEAM for ECH/ECCD computations; TGLF and QuaLiKiz for quasi-linear computation of turbulence-driven transport; TORIC for ICRF computations; STRAHL for impurity transport studies; NEO for neoclassical transport calculations (libraries are to be requested to the respective developer);
- A routine to create Astra restart files has been developed;
- A way to comment entire blocks in the model file has been introduced: in the usual case, one uses the symbol "!" to comment a line, like

```
!CV7 = 1;
```

means that ASTRA will ignore the line above during compilation. However to comment multiple lines can be painful and not very efficient. So the symbol "%" has been introduced. By using it, ASTRA ignores everything between a pair of "%" symbol, as in the following example:

```
%
```

```
CV7 = 1;
```

```
PE = POH;
```

```
PI=0.;
```

```
%
```

the entire block above will be ignored during compilation.

- A way to select entire blocks inside the model file:

The symbol "#" has been introduced. By using it, ASTRA uses only what is inside the block

delimited by a pair of ”#” symbol, with the first appearance followed by a single digit or letter identifier. The latter shall appear at the beginning of the model file. For example:

```
#1
```

```
CV7 = 1;
```

```
PE = POH;
```

```
PI=0.;
```

```
#
```

the above entire block above will be the only one used during compilation if the beginning of the model file contains ”1”. Everything else is ignored that is inside other blocks delimited by # . However, what is outside of these blocks, is still taken into account. This new symbol thus allows to have several types of physics models inside a single model file, selectable by changing the digit or letter identifier at the beginning of the model file (first digit or letter of the model file), with having some common commands still read (for example plotting or common definitions).

- A new Newton-scheme solver that allows for faster convergence to an absolute steady-state

10 Foreseen upgrades for next versions

As the code is in continue development, here are listed some foreseen upgrades for future versions. The order is almost as it should proceed chronologically (mostly).

- A python-based version of the model writing routines, to improve flexibility (in progress with G. Tardini);
- A version of ASTRA as a subroutine (long-term project).